Search and CSP Approaches for a Simplified Connect-4 Puzzle

CPS 367 Final Project Presentation

Student: Thanh Vu

Problem Selection

Domain Choice

Selected Connect-4 as problem domain per course instructions for simple game playing with small boards.

Simplification

Full two-player Connect-4 has ~4.5 trillion states for 6x7 board.

Simplified to single-player puzzle: add fewest pieces to form 4-in-a-row.

Board Size

Used 4x6 board for manageable state space: 5^6 = 15,625 states. Large enough to test efficiency, small enough for quick experiments.

Task Environment Definition

States & Actions

States: Tuple of 6 integers representing column heights (0-4 each).

Actions: Drop piece in column c if height[c] < 4, incrementing by 1.

Transition & Goal

Transition: Update heights tuple by adding 1 to selected column.

Goal Test: Build 2D board from heights, check for 4 consecutive 1s in any direction.

Cost & Start

Path Cost: 1 per action (minimizes added pieces for optimal solution).

Sample Start: Empty board, Heights = (0, 0, 0, 0, 0, 0) on 4x6 grid.

Search Implementation Overview

Implemented in single Python file (connect4_puzzle.py) using basic structures from class examples. States as tuples for hashing in visited sets.

Helper Functions

build_board(heights): Create 2D list, fill bottom-up. has_connect(board): Check for 4-in-a-row. get_successors(heights): Generate new states.

Search Algorithms

Uninformed: BFS for shortest path with uniform costs. Informed: A* with priority queue (f = g + h).

Measurements

Tracked cost, nodes expanded, and runtime using time module for performance analysis.

Search Pseudo Code

BFS Algorithm

```
Initialize queue with (start, 0)
visited set with start

While queue not empty:
  Dequeue state, cost
  Increment nodes_exp
  If goal test passes:
    Return cost, nodes_exp
  For each successor:
    If not visited:
      Add to visited and queue
      with cost + 1
```

A* Algorithm

```
Initialize priority queue
with (h(start), 0, start)

While queue not empty:
 Pop f, g, state
 Increment nodes_exp
 If visited, continue
  Add to visited
 If goal test passes:
  Return g, nodes_exp
 For each successor:
  Push (g+1+h(succ), g+1, succ)
```

Heuristic Design & Analysis

1

Zero Heuristic (h=0)

Baseline that turns A* into uniform-cost search. Always admissible as h ≤ true cost.

2

Max Connect Heuristic

h(state) = 4 - max_connect_length. Measures progress to goal; prioritizes states close to winning (e.g., 3-in-a-row gives h=1).

3

Admissibility

Max connect doesn't overestimate as each drop adds at most 1 to length. Domain-specific heuristic reduces nodes expanded.

CSP Formulation



Variables & Domains

Variables: h0 to h5 (final column heights). **Domains:** {0, 1, ..., 4} for each hi from empty start.



Constraints

Global disjunction ensuring at least one 4-in-a-row: horizontal (all heights ≥ r+1), vertical (height[c] ≥ 4), diagonals (similar checks).



Search vs CSP

Search better for optimization (minimal pieces via costs/heuristics). CSP for satisfiability but needs extensions for min sum(hi).

Experimental Evaluation

Algorithms Compared

BFS, A* with h=0, A* with max connect heuristic.

Hypotheses

All find same optimal cost (admissible). Max connect expands fewest nodes, fastest runtime. Harder instances expand more nodes.

Setup

Test case: Empty board, Heights = (0,0,0,0,0,0), goal connect=4. Measured cost, nodes expanded, runtime. Deterministic, one run per algorithm on Python 3.10.

Results and Analysis

BFS	4	85	0.0009
A* (h=0)	4	170	0.0020
A* (max connect)	4	38	0.0010

Key Findings:

- Results Table (Connect=4, Empty Board)
- Analysis: Optimal costs consistent; max connect reduces nodes >50% vs BFS by prioritization
- h=0 expands more due to no guidance; heuristics scale better for larger problems
- Bottlenecks: Board building and connect checks; heuristics scale better for larger problems
- No surprises; confirms domain heuristic efficiency

Critical Reflection

Successes & Challenges

Successes: Simplification made implementation easy. Heuristic improved performance clearly.

Challenges: Debugging connect checks (diagonals). Defining precise CSP constraints.

Extensions & Learning

Extensions: Larger boards . Full CSP solver comparison.

Learning: Reinforced search vs CSP trade-offs for optimization vs satisfiability.

Thank you for listening!